

提高组 CSP-S 2024 初赛模拟卷 4

一、单项选择题（共 15 题，每题 2 分，共计 30 分；每题有且仅有一个正确选项）

1. 运行如下代码的输出结果是（ ）。

```
char c = '0' + ' ' ;  
cout << c << endl;
```

- A. 0 B. P C. 48 D. 80

2. 以下代码段的功能是（ ）。

```
#include <iostream>  
using namespace std;  
int lowbit(int x) {  
    return (x & -x);  
}  
int main() {  
    int n, res = 0;  
    cin >> n;  
    while (n)  
        n -= lowbit(n), res++;  
    cout << res;  
    return 0;  
}
```

- A. 求该数转为二进制数以后各位上 1 的总数
B. 求该数转为二进制的反码
C. 求该数转为二进制的补码
D. 求该数转为二进制的原码
3. 前缀表达式 $- * + 1\ 34\ 5\ /\ 56\ 7$ 的后缀表达式为（ ）。
- A. $1 + 34 * 5 - 56 /\ 7$
B. $- * + 1\ 34\ 5 /\ 56\ 7$
C. $1\ 34 + 5 * 56\ 7 /\ -$
D. $1\ 34\ 5 * + 56\ 7 /\ -$

4. 在数组 $A[x]$ 中, 若存在 $(i < j) \ \&\& \ (A[i] > A[j])$, 则称 $(A[i], A[j])$ 为数组 $A[x]$ 的一个逆序对。对于序列 7, 4, 1, 9, 3, 6, 8, 5, 其中有 () 个逆序对。
A. 10 B. 11 C. 12 D. 13
5. 以下哪个操作不属于 STL 中双端队列 (deque) 的操作函数? ()
A. front B. back C. top D. erase
6. KMP 算法不属于下面哪位人士参与研究的算法? ()
A. Knuth B. McDonald C. Morris D. Pratt
7. 命题 “ $P \rightarrow Q$ ” 可读作 P 蕴含 Q , 其中 P 、 Q 是两个独立的命题。只有当命题 P 成立而命题 Q 不成立时, 命题 “ $P \rightarrow Q$ ” 的值才为 false, 其他情况均为 true。与命题 “ $P \rightarrow Q$ ” 等价的逻辑关系式是 ()。
A. $\neg P \vee Q$ B. $P \wedge Q$ C. $\neg (P \vee Q)$ D. $\neg (\neg Q \wedge P)$
8. 有一个 5×5 的棋盘, 棋盘左下格有一枚棋子, 我们每次可以将棋子向右移动一格, 也可以向上移动一格, 但棋盘的某些方格不可以放置棋子 (如下图所示, 画 $\times$ 的方格表示不可以放置棋子), 则棋子从左下格到右上格一共存在 () 条路径。

	$\times$			
			$\times$	
		$\times$		
o				

- A. 10 B. 11 C. 12 D. 13
9. 关于图论, 下面的说法哪个是正确的? ()
A. 欧拉路有一个奇点
B. 欧拉回路有两个奇点
C. 对于一个无向图, 如果任意一对顶点都是连通的, 则此图被称为连通图
D. 不存在割点的无向图称为 “2-边连通图”

二、阅读程序（程序输入不超过数组或字符串定义的范围；判断题正确填√，错误填×；除特殊说明外，判断题每题 1.5 分，选择题每题 3 分，共计 40 分）

(1)

```
01 #include <iostream>
02 using namespace std;
03
04 const int N = 100500;
05 const int Mod = 1000000007;
06 int n, m, j, p[N], f[N];
07 char s[N], t[N];
08
09 int main() {
10     scanf("%s", s+1); scanf("%s", t+1);
11     n = strlen(s+1); m = strlen(t+1);
12     j = 0; p[1] = 0;
13     for (int i = 1; i < m; ++i) {
14         while (j > 0 && t[j+1] != t[i+1]) j = p[j];
15         if (t[j+1] == t[i+1]) ++j;
16         p[i+1] = j;
17     }
18     j = 0; f[0] = 1;
19     for (int i = 1; i <= n; ++i) {
20         f[i] = f[i-1];
21         while (j > 0 && s[i] != t[j+1]) j = p[j];
22         if (s[i] == t[j+1]) ++j;
23         if (j == m) {
24             j = p[j];
25             (f[i] += f[i-m]) %= Mod;
26         }
27     }
28     printf("%d\n", f[n]);
29     return 0;
30 }
```

■ 判断题

16. 若输入数据导致 $m > n$ ，则输出一定为 1。 ()
17. 若输入数据导致 $m = 1$ ，则输出一定为 2 的整次幂。 ()

18. 去掉第 12 行, 不影响程序的输出结果。 ()
19. 若将本程序改为可读入多组数据, 则并不用增加新的初始化代码。 ()

■ 选择题

20. 若输入为 "xxxxx xx", 则输出为 ()。
- A. 5 B. 8 C. 10 D. 15
21. 输入数据的两个字符串长度分别为 N 、 M , 则本程序的时间复杂度为 ()。
- A. $O(M)$ B. $O(N)$ C. $O(M+N)$ D. $O(MN)$

(2)

```
01 #include <iostream>
02
03 const int mod=1e4;
04 int n;
05 int ans;
06
07 inline int qpow(int a, int b) {
08     int res = 1;
09     for (; b; b >>= 1, a = 1LL * a * a % mod)
10         if (b & 1) res = 1LL * res * a % mod;
11     return res;
12 }
13
14 inline void add(int & x, const int & y) {
15     x += y;
16     if (x >= mod) x -= mod;
17 }
18
19 inline int calc(int n) {
20     int res = 0;
21     int c = 1, x = 1;
22     for (; x * 10 - 1 <= n; ++c, x *= 10)
23         add(res, 9LL * x * c % mod);
24     add(res, 1LL * c * (n-x+1) % mod);
25     return res;
26 }
27
```

```
28 int main() {
29     std::cin >> n;
30     if (n == 1) return puts("4"), 0;
31     ans = n % mod;
32     add(ans, 1LL * qpow(2, n-1) * n % mod);
33     add(ans, (calc(n) - 1 + mod) % mod);
34     add(ans, 1LL * qpow(2, n-1) * calc(n) % mod);
35     add(ans, (qpow(2, n) - 1 + mod) % mod);
36     add(ans, 2 * (n-1) % mod);
37     std::cout << ans << std::endl;
38     return 0;
39 }
```

■ 判断题

22. 第 14 行的两个 "8" 符号的作用一致。 ()
23. 第 16 行等价于把 x 对 mod 取模, 只是更慢。 ()
24. 输入为 5 的时候, 输出为 209。 ()
25. 输入每增加 1, 输出扩大 2 倍以上。 ()

■ 选择题

26. 输入为 0 的时候, 程序会 ()。
- A. 输出乱码 B. 死循环 C. 输出 "0" D. 输出 "-1"
27. 本程序的时间复杂度为 ()。
- A. $O(\log N)$ B. $O(\log \log N)$ C. $O(N)$ D. $O(\log_2 N + \log N)$

(3)

```
01 #include <iostream>
02 using namespace std;
03 typedef long long i64;
04
05 const int NN = 1e7+5;
06 i64 ans, l, r;
07 bool s[NN];
08 int p[NN], o[NN], cnt;
09
10 inline void sss() {
```

```
11  o[1] = 1;
12  for (int i = 2; i <= r; ++i) {
13      if (!s[i])
14          o[p[++cnt] = i] = i;
15      for (int j = 1; j <= cnt && p[j] * i <= r; ++j) {
16          s[p[j] * i] = true;
17          o[p[j] * i] = p[j];
18          if (i % p[j] == 0) break;
19      }
20  }
21 }
22
23 inline void answer() {
24     sss();
25     for (int i = l; i <= r; ++i)
26         if (s[i]) ans += i/o[i];
27     cout << ans << endl;
28 }
29
30 int main() {
31     cin >> l >> r;
32     answer();
33 }
```

■ 判断题

28. (2 分) 本题使用了埃氏筛法。 ()
29. (2 分) 如果输入数据 l 比 r 大, 那么程序会报错或者发生死循环。 ()
30. (2 分) 本程序可以处理的最大 r 是 100000000。 ()
31. (2 分) 在确保 $l < r$ 的前提下, l 的大小不影响程序的运行速度。 ()

■ 选择题

32. (4 分) 当输入为 "6 27" 时, 输出为 ()。
- A. 115 B. 116 C. 117 D. 118
33. (4 分) 本程序的时间复杂度为 ()。
- A. $O(\log N)$ B. $O(N)$ C. $O(\log \log N)$ D. $O(N \log \log N)$

三、完善程序（单选题，每小题 3 分，共计 30 分）

- (1) 输入一系列短棍的长度，它们是由一些等长的木棒截断得来的。请输出原始木棒的最小可能长度。截断后的短棍不超过 64 根，且长度不超过 50。本程序使用搜索+剪枝的办法来解决该问题，共使用 4 个剪枝。

比如，输入数据为：

9

5 2 1 5 2 1 5 2 1

输出数据为：

6

拼凑方案为：

5+1, 5+1, 5+1, 2+2+2

```
01 #include <iostream>
02 using namespace std;
03
04 int a[100], v[100], n, len, cnt;
05 bool dfs(int stick, int cab, int last) {
06     if (stick > cnt) return true;
07     if (cab == len) return dfs(stick + 1, 0, 1);
08     int fail = 0;
09     for ( ① )
10         if (!v[i] && cab + a[i] <= len && fail != a[i]) {
11             v[i] = 1;
12             if (dfs(stick, cab + a[i], i+1)) return true;
13             ②;
14             v[i] = 0;
15             if (③||④) return false;
16         }
17     return false;
18 }
19
20 int main() {
21     while (cin >> n && n) {
22         int sum = 0, val = 0;
23         for (int i = 1; i <= n; i++) {
24             scanf("%d", &a[i]);
25             sum += a[i]; val = max(val, a[i]);
```



```
26     }
27     sort(a + 1, a + n + 1);
28     reverse(a + 1, a + n + 1);
29     for (len = val; len <= sum; len++) {
30         if (sum % len) continue;
31         cnt = sum / len;
32         memset(v, 0, sizeof(v));
33         if (⑤) break;
34     }
35     cout << len << endl;
36 }
37 return 0;
38 }
```

34. ①处应填 ()。

- A. `int i = n; i >= last; i--`
- B. `int i = last; i <= n; i++`
- C. `int i = last + 1; i <= n; i++`
- D. `int i = n; i >= last + 1; i--`

35. ②处应填 ()。

- | | |
|---------------------------------|-----------------------------|
| A. <code>fail += a[i]</code> | B. <code>a[i] = fail</code> |
| C. <code>fail = cab+a[i]</code> | D. <code>fail = a[i]</code> |

36. ③处应填 ()。

- | | |
|---------------------------------|-----------------------------|
| A. <code>cab == 0</code> | B. <code>cab == a[i]</code> |
| C. <code>a[last] == fail</code> | D. <code>last == 0</code> |

37. ④处应填 ()。

- | | |
|--------------------------------------|-----------------------------------|
| A. <code>cab + a[i] <= len</code> | B. <code>a[i] == fail</code> |
| C. <code>cab + a[i] == len</code> | D. <code>a[i] > cab / 2</code> |

38. ⑤处应填 ()。

- | | |
|------------------------------|------------------------------|
| A. <code>dfs(0, 0, 1)</code> | B. <code>dfs(1, 0, 1)</code> |
| C. <code>dfs(0, 0, 0)</code> | D. <code>dfs(1, 0, 0)</code> |

- (2) 给定一个长度不超过 17 的字符串及整数 K , 字符串由 0~9 组成, 且无前导 0。求使用其中的数字排列成无前导 0 且能被 17 整除的第 K 小的数。 $K \leq 17!$ 。

比如, 输入为 2242223 2, 输出为 2242232。

使用状态压缩 DP, $f[n][val][State]$ 表示从个位 (个位为第 1 位) 填到第 n 位时, 还没有用过的数字对应的状态编号为 $State$, 还没有用过的数字构成的数对 17 取模的结果为 val , 且整体为 17 倍数的情况下的合法方案数 (现在的 n 位数加上 $val \times 10^n$ 之后是 17 的倍数)。

```
01 #include <iostream>
02 using namespace std;
03
04 typedef long long ll;
05 const int N=20, M=20005;
06 int cnt[N]; char s[N];
07 ll f[N][N][M], d[N], K;
08
09 inline ll dfs(int k, int val, int sta) {
10     ①;
11     if (~f[k][val][sta]) return f[k][val][sta];
12     ll res = 0;
13     for (int i = 0; i <= 9; ++i) {
14         int lim = sta/d[i] % (cnt[i] + 1);
15         if (lim == cnt[i]) continue;
16         res += ②;
17     }
18     return f[k][val][sta] = res;
19 }
20
21 int main() {
22     ③;
23     scanf("%s", s+1); cin >> K;
24     int len = strlen(s+1);
25     for (int i = 1; i <= len; ++i)
26         ++cnt[s[i]-'0'];
27     d[0] = 1;
28     for (int i = 1; i <= 9; ++i)
29         d[i] = ④;
30     int sta = 0, val = 0; ll now = 0;
31     for (int i = len; i >= 1; --i)
32         for (int j = i == len?1:0; j <= 9; ++j) {
```

```
33         int lim = ⑤;
34         if (lim == cnt[j]) continue;
35         ll tmp = dfs(i-1, (val*10+j)%17, sta+d[j]);
36         if (now + tmp >= K) {
37             putchar(j+'0');
38             val = (val * 10 + j) % 17;
39             sta += d[j];
40             break;
41         }
42         now += tmp;
43     }
44     cout << endl;
45     return 0;
46 }
```

39. ①处应填 ()。

- | | |
|-----------------------|------------------------|
| A. if (k) return !val | B. if (!k) return !val |
| C. if (!k) return val | D. if (k) return val |

40. ②处应填 ()。

- A. dfs(k+1, (val - i * 10) % 17, sta + d[i])
B. dfs(k-1, (val * 10 + i) % 17, sta - d[i])
C. dfs(k+1, (val - i * 10) % 17, sta - d[i])
D. dfs(k-1, (val * 10 + i) % 17, sta + d[i])

41. ③处应填 ()。

- | | |
|------------------------------|------------------------------|
| A. memset(f, 255, sizeof(f)) | B. memset(f, 0, sizeof(f)) |
| C. memset(f, 127, sizeof(f)) | D. memset(f, 128, sizeof(f)) |

42. ④处应填 ()。

- | | |
|--------------------------|------------------------|
| A. d[i-1] * (cnt[i-1]) | B. d[i-1] * (cnt[i]) |
| C. d[i-1] * (cnt[i-1]+1) | D. d[i-1] * (cnt[i]+1) |

43. ⑤处应填 ()。

- | | |
|------------------------------|----------------------------|
| A. sta / d[j] % (cnt[j]+1) | B. sta / d[j-1] % (cnt[j]) |
| C. sta / d[j-1] % (cnt[j]+1) | D. sta / d[j] % (cnt[j]) |